



Redes Neurais

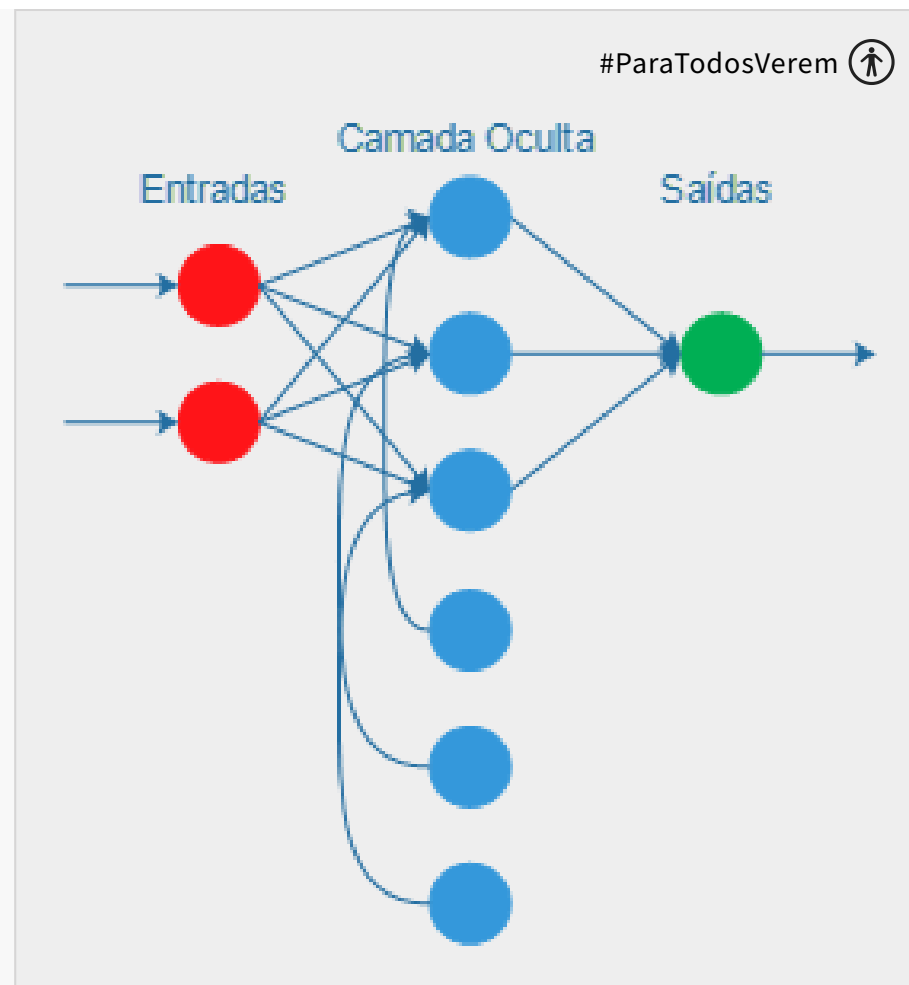
UNIDADE 06

Rede Neural Temporal (RNT)

| Conceitos Gerais

Uma Rede Neural Temporal (RTN) é uma arquitetura de rede neural que tem a capacidade de processar e analisar dados sequenciais ao longo do tempo. Ao contrário das redes neurais tradicionais, que tratam cada entrada de dados de forma isolada, a RNT possui conexões recorrentes que permitem que informações anteriores sejam armazenadas e utilizadas para tomar decisões em momentos futuros. A Figura 1, apresentada a seguir, mostra a arquitetura básica de uma RNT.

Fig. 1 – Arquitetura básica de uma Rede Neural Temporal



Fonte: Autor, 2023.

A RNT é capaz de capturar a dependência temporal nos dados, o que a torna especialmente adequada para lidar com problemas em que a ordem e a relação entre os elementos são importantes, como séries temporais, linguagem natural, processamento de sinais etc. Ela utiliza unidades de memória chamadas de neurônios e sinapses temporais, que permitem armazenar informações ao longo do tempo e processá-las em etapas subsequentes. Essa capacidade de memória temporal é uma característica fundamental da RNT e a diferencia das demais arquiteturas.

A RNT tem uma ampla gama de aplicações em diversos setores. Alguns exemplos de aplicações práticas incluem:

| Aplicações práticas em diferentes tipos de problema

- Processamento de linguagem natural: a RNT pode ser usada para modelar a estrutura temporal de sequências de palavras em textos, melhorando a qualidade de tradução automática, análise de sentimentos, geração de texto e resumos automáticos.
- Previsão de séries temporais: a capacidade da RNT de capturar dependências temporais é especialmente útil para prever tendências em séries temporais, como previsão de vendas, previsão de demanda de energia, previsão de preços de ações etc.
- Processamento de sinais: a RNT pode ser aplicada em problemas de processamento de áudio e vídeo, como reconhecimento de fala, processamento de música, análise de vídeo e detecção de eventos em sinais temporais.
- Análise de dados biomédicos: a RNT tem sido utilizada em aplicações biomédicas, como análise de sinais cerebrais, previsão de eventos médicos, diagnóstico de doenças e monitoramento de pacientes.
- Robótica e controle de sistemas: a RNT pode ser empregada no controle de sistemas dinâmicos, como robôs, sistemas de energia e processos industriais, permitindo o aprendizado e a adaptação em tempo real.

| Arquitetura da Rede Neural Temporal

Vamos explorar agora os componentes essenciais de uma RNT. Veremos os neurônios e sinapses temporais, a estrutura e como ocorre o fluxo de informação.

- **Neurônios e sinapses temporais:** são os blocos de construção fundamentais de uma RNT.
 - o Neurônios temporais: são unidades de processamento que possuem uma propriedade única chamada de memória temporal. Eles são capazes de armazenar informações do passado e atualizá-las ao longo do tempo. Cada neurônio temporal possui um estado interno chamado de potencial de memória, que é atualizado a cada instante de tempo.

- o Sinapses temporais: as sinapses temporais são as conexões entre os neurônios em uma RNT. Cada sinapse possui um peso associado que determina a importância da informação transmitida de um neurônio para outro. Esses pesos são atualizados durante o treinamento da rede para melhorar seu desempenho.
- **Estrutura da RNT:** a estrutura básica é uma rede neural recorrente, que consiste em uma série de neurônios conectados em camadas. A principal característica das redes neurais recorrentes é a presença de conexões retroalimentadas, permitindo que a informação flua tanto para frente quanto para trás na rede.
 - o Camadas ocultas: uma RNT pode ter uma ou mais camadas ocultas, que são compostas por neurônios temporais. Cada camada oculta processa as informações recebidas e gera uma saída que é passada para a próxima camada.
 - o Camada de saída: a camada de saída da RNT recebe as informações processadas pelas camadas ocultas e produz a saída final da rede. Dependendo do problema em questão, a camada de saída pode ter diferentes configurações, como neurônios de classificação, neurônios de regressão ou neurônios de geração de texto, por exemplo.
- **Fluxo de informação em uma RNT:** ocorre ao longo do tempo, e a principal característica é a recorrência das conexões.
 - o Etapas de tempo: em uma RNT, o processamento ocorre em etapas de tempo discretas. A cada etapa de tempo, os neurônios temporais atualizam seu estado interno com base nas informações recebidas e na memória temporal armazenada. Essa atualização é realizada usando as sinapses temporais e os pesos associados.
 - o Retroalimentação temporal: uma das principais características da RNT é a retroalimentação temporal, que permite que as informações do passado influenciem o processamento no presente e futuro. Isso é alcançado por meio das conexões retroalimentadas, que permitem que as informações fluam entre as camadas e etapas de tempo.
 - o Propagação do erro: durante o treinamento da RNT, é necessário calcular o erro cometido pela rede e ajustar os pesos das sinapses para minimizá-lo. A propagação do erro ocorre ao longo das etapas de tempo, permitindo que o erro seja retroalimentado na rede para atualizar os pesos de forma a melhorar o desempenho.

| Treinamento de uma RNT

Veremos agora os algoritmos de aprendizado utilizados nas RNTs, os problemas comuns e desafios enfrentados no treinamento dos modelos e algumas técnicas de otimização.

- **Algoritmos de aprendizado:** existem vários algoritmos de aprendizado que podem ser utilizados no treinamento de uma RNT. Os mais comuns incluem:
 - o *Backpropagation Through Time* (BPTT): é uma extensão da retropropagação (backpropagation) usada em outras redes neurais recorrentes. Nesse algoritmo, o erro é propagado ao longo das etapas de tempo, permitindo que os pesos das sinapses sejam atualizados de acordo com o gradiente do erro.
 - o *Truncated Backpropagation Through Time* (TBPTT): devido à retropropagação ao longo de várias etapas de tempo, o BPTT pode se tornar computacionalmente custoso. O TBPTT é uma técnica que divide a sequência temporal em segmentos menores, reduzindo o tempo de processamento e limitando o alcance da retropropagação.
 - o *Real-Time Recurrent Learning* (RTRL): é um algoritmo que calcula o gradiente em tempo real para atualizar os pesos da RNT. Ele usa equações de diferenças parciais para estimar o gradiente, tornando-o adequado para aplicações em tempo real.
 - o *Long Short-Term Memory* (LSTM): um tipo especial de célula de memória recorrente amplamente utilizado na RNT. O LSTM é projetado para resolver o problema do desvanecimento ou explosão do gradiente em redes neurais temporais. Este algoritmo é utilizado no código fornecido para o desenvolvimento do exercício proposto nesta unidade.
- **Problemas comuns e desafios no treinamento de RNT:** o treinamento de RNTs pode apresentar desafios específicos devido à sua natureza temporal e à recorrência das conexões. Alguns problemas comuns enfrentados no treinamento de RNTs incluem:

- o *Vanishing/Exploding gradient*: durante o treinamento, os gradientes podem diminuir exponencialmente (*vanishing gradient*) ou crescer exponencialmente (*exploding gradient*). Isso pode dificultar a convergência da rede ou levar a instabilidades numéricas. Técnicas como gradiente truncado (*truncated gradient*) e inicialização adequada dos pesos podem ajudar a mitigar esse problema.
- o *Overfitting e underfitting*: assim como em outras arquiteturas de rede neural, o *overfitting* (sobreajuste) e o *underfitting* (subajuste) são desafios no treinamento de RNTs. O *overfitting* ocorre quando a rede se ajusta muito bem aos dados de treinamento, mas não generaliza bem para novos dados. O *underfitting* ocorre quando a rede não é capaz de capturar informações importantes dos dados. Técnicas de regularização, como dropout e regularização L1/L2, podem ser aplicadas para lidar com esses problemas.
- **Técnicas de otimização**: a escolha do algoritmo de otimização adequado é crucial para o treinamento eficiente de uma RNT.
 - o *Stochastic Gradient Descent* (SGD): pode ser usado para otimizar os pesos da rede.
 - o Adam e RMSprop: também apresentam bons resultados na utilização da RNT.

| Parâmetros ajustáveis do modelo

- Janela de previsão (*janela_prev*): define quantos dados históricos o modelo utilizará para prever o próximo valor. Por exemplo, *janela_prev* = 5, significa que o modelo vai considerar cinco valores anteriores para prever o sexto, durante o treinamento.
- Função de perda ou função custo: define a evolução de determinação dos pesos das sinapses durante o treinamento. Possui influência direta no desempenho do modelo. São elas:
 - o `mean_squared_error`
 - o `mean_absolute_error`
 - o `categorical_crossentropy`

- o `binary_crossentropy`
- Otimizador: otimiza a evolução dos pesos das sinapses durante o treinamento, possuindo também influência direta no desempenho do modelo. São eles:
 - o `sgd`
 - o `adam`
 - o `RMSdrop`
- Número de neurônios da camada LSTM (`neurônios_LSTM`): define quantos neurônios serão utilizados para trocar informações da série histórica durante o treinamento.
- Número de neurônios da camada densa (`neuronios_dense`): define quantos neurônios serão utilizados na camada densa, enviando as informações para a camada de saída.
- Número de épocas de treinamento (`epocas`): define quantas vezes o modelo irá passar pelo conjunto de dados a cada treinamento. Quanto maior for o número de épocas, maior a chance da rede aprender características específicas da base de dados. Entretanto, um número muito alto pode fazer com que ela não consiga generalizar as previsões para dados futuros que difiram dos dados utilizados durante o treinamento.
- Tamanho do lote (`lote`): este parâmetro indica quantos subconjuntos de dados serão utilizados durante o treinamento a cada época. Por exemplo, se tivermos 100 dados históricos e o tamanho do lote for definido para 2, o modelo utilizará 50 dados em cada lote; possuirá, portanto, duas baterias de treinamento a cada época, cada uma com metade dos dados de entrada. Portanto, é sempre bom verificar durante o pré-processamento a quantidade de dados disponível e o comportamento da série temporal (variações, repetições sazonais etc.).

Durante a atividade formativa proposta para esta unidade, procure realizar o máximo de combinações possível entre os parâmetros ajustáveis. A cada término de treinamento, anote os valores dos parâmetros escolhidos e os resultados obtidos, para que possa compará-los com os próximos que serão realizados. E, após concluir que o desempenho de um determinado conjunto de configurações do modelo é mais

satisfatório do que os demais, você poderá retornar para o respectivo ajuste e apresentar os resultados.



DICA

Utilize como análise de desempenho do modelo as informações finais apresentadas no *Console* da IDE Spyder:

Erro Médio Quadrático (MSE): Quanto menor, melhor.

Erro Médio Absoluto (MAE): Quanto menor, melhor.

Coefficiente de Determinação (R^2): Quanto mais próximo de 1, melhor.

| Conclusão

Ao longo desta unidade, você obteve uma sólida compreensão dos fundamentos da RNT, suas aplicações práticas e os desafios enfrentados nessa área. Vimos também a importância de ajustar os parâmetros de forma adequada, para um bom desempenho do modelo. Você poderá utilizar esse conhecimento para aplicar a estrutura RNT em diversos campos, solucionando diferentes tipos de problema.

| Referências

HAYKIN, S. **Redes neurais**: Princípios e prática. 2. ed. Porto Alegre, RS: Bookman, 2007.

FACELI, K. et al. **Inteligência artificial**: Uma abordagem de aprendizagem de máquina. Rio de Janeiro, RJ: LTC, 2011.

SILVA, I. N.; SPATTI, D. H.; FLAUZINO, R. A. **Redes neurais artificiais**: Para engenharia e ciências aplicadas. São Paulo, SP: Artliber, 2010. p. 399.

